

PCAP Programming

과제 내용

-패킷의 정보 출력

- Ethernet Header(src mac, dst mac)
- IP Header(src ip, dst ip)
- TCP Header(src port, dst port)
- HTTP Message
- TCP만 대상으로

개발 환경

- OS: Ubuntu 24.04 LTS(WSL)
- Compiler: GCC Compiler
- Library: libpcap-dev 패키지

과제 목표

-C/C++ 기반 PCAP API를 활용하여 패킷을 캡처하고, Ethernet Header, IP Header, TCP Header의 주소 정보, HTTP Message를 출력한다.

header.h

```
1  /* Ethernet header */
2  struct ethheader {
3      u_char ether_dhost[6]; /* destination host address */
4      u_char ether_shost[6]; /* source host address */
5      u_short ether_type;    /* IP? ARP? RARP? etc */
6  };
7
8  /* IP Header */
9  struct ipheader {
10     unsigned char iph_ihl:4, //IP header length
11     unsigned char iph_ver:4; //IP version
12     unsigned char iph_tos; //Type of service
13     unsigned short int iph_len; //IP Packet length (data + header)
14     unsigned short int iph_ident; //Identification
15     unsigned short int iph_flag:3, //Fragmentation flags
16     unsigned short int iph_offset:13; //Flags offset
17     unsigned char iph_ttl; //Time to Live
18     unsigned char iph_protocol; //Protocol type
19     unsigned short int iph_checksum; //IP datagram checksum
20     struct in_addr iph_sourceip; //Source IP address
21     struct in_addr iph_destip; //Destination IP address
22 };
```

```

24  /* TCP Header */
25  struct tcpheader {
26      u_short tcp_sport;          /* source port */
27      u_short tcp_dport;          /* destination port */
28      u_int tcp_seq;              /* sequence number */
29      u_int tcp_ack;              /* acknowledgement number */
30      u_char tcp_offx2;           /* data offset, rsvd */
31      #define TH_OFF(th)          (((th)->tcp_offx2 & 0xf0) >> 4)
32      u_char tcp_flags;
33      #define TH_FIN  0x01
34      #define TH_SYN  0x02
35      #define TH_RST  0x04
36      #define TH_PUSH 0x08
37      #define TH_ACK  0x10
38      #define TH_URG  0x20
39      #define TH_ECE  0x40
40      #define TH_CWR  0x80
41      #define TH_FLAGS (TH_FIN|TH_SYN|TH_RST|TH_ACK|TH_URG|TH_ECE|TH_CWR)
42      u_short tcp_win;            /* window */
43      u_short tcp_sum;            /* checksum */
44      u_short tcp_urp;            /* urgent pointer */
45  };

```

sniff.c

```

1  #include <stdlib.h>
2  #include <stdio.h>
3  #include <pcap.h>
4  #include <arpa/inet.h>
5  #include <ctype.h>
6  #include "myheader.h"
7
8  void got_packet(u_char *args, const struct pcap_pkthdr *header,
9                  const u_char *packet)
10 {
11     struct ethheader *eth = (struct ethheader *)packet;
12
13     if (ntohs(eth->ether_type) == 0x0800) { // 0x0800 is IP type
14         struct ipheader *ip = (struct ipheader *)
15             (packet + sizeof(struct ethheader));
16         //TCP만 대상으로
17         if (ip->iph_protocol == IPPROTO_TCP) {
18             //IP 헤더 길이 계산
19             int ip_header_len = ip->iph_ihl * 4;
20             //TCP 헤더 파싱
21             struct tcpheader *tcp = (struct tcpheader *)((u_char*)ip + ip_header_len);
22             //TCP 헤더 길이 계산
23             int tcp_header_len = TH_OFF(tcp) * 4;
24             //HTTP Message 시작 위치 계산
25             u_char *data = (u_char *)tcp + tcp_header_len;

```

패킷 수신(11~14라인)

패킷이 수집되면 가장 먼저 최하위 계층인 Ethernet 헤더 구조체로 형변환한다.

Ethernet 헤더의 프로토콜 타입(ether_type)이 0x0800인 경우에만 다음 단계로 진행하며, packet 시작 주소에 Ethernet 헤더의 고정 크기(sizeof(struct ethheader))를 더해 IP

헤더의 시작 위치를 찾아낸다.

TCP 프로토콜 필터링 및 길이 계산(17~21라인)

IP 헤더의 프로토콜 필드(iph_protocol)를 확인하여 TCP 패킷(6)만 대상으로 필터링한다.

IP 헤더의 크기는 옵션에 따라 달라질 수 있는 가변 길이 -> 헤더 길이 필드인 iph_ihl 값에 4를 곱해 실제 IP 헤더 크기(ip_header_len)를 구한다. 이후 IP 헤더 시작 주소에 이 길이를 더해 TCP 헤더의 정확한 시작 위치를 계산한다.

TCP 헤더 크기 및 HTTP 계산(23~25 라인)

TCP 헤더 또한 가변 길이 가짐 -> myheader.h에 정의된 TH_OFF 매크로를 사용해 TCP 헤더의 데이터 오프셋(상위 4비트 값)을 구한 뒤 4를 곱해 실제 TCP 헤더 크기(tcp_header_len)를 산출한다.

TCP 헤더 시작 주소에 계산된 tcp_header_len을 더해 주면, 최종적으로 Application 계층의 데이터인 HTTP Message의 시작 주소(data)를 가리키게 된다.

```
27 //데이터 크기 계산
28 int total_header_len = sizeof(struct ethheader) + ip_header_len + tcp_header_len;
29 int data_len = header->caplen - total_header_len;
30 //정보 출력
31 printf("=====PACKET=====\\n");
32 //Ethernet Header
33 printf("src mac: %02x:%02x:%02x:%02x:%02x:%02x\\n",
34        eth->ether_shost[0], eth->ether_shost[1], eth->ether_shost[2],
35        eth->ether_shost[3], eth->ether_shost[4], eth->ether_shost[5]);
36 printf("dst mac: %02x:%02x:%02x:%02x:%02x:%02x\\n",
37        eth->ether_dhost[0], eth->ether_dhost[1], eth->ether_dhost[2],
38        eth->ether_dhost[3], eth->ether_dhost[4], eth->ether_dhost[5]);
39 //IP Header
40 printf("src ip: %s\\n", inet_ntoa(ip->iph_sourceip));
41 printf("dst ip: %s\\n", inet_ntoa(ip->iph_destip));
42 //TCP Header
43 printf("src port: %d\\n", ntohs(tcp->tcp_sport));
44 printf("dst port: %d\\n", ntohs(tcp->tcp_dport));
45 //HTTP Message
46 printf("HTTP Message\\n");
47 if (data_len > 0){
48     for (int i = 0; i < data_len; i++){
49         if(isprint(data[i]) || data[i] == '\\n' || data[i] == '\\r'){
50             putchar(data[i]);
51         }
52         else{
53             putchar('.');
54         }
55     }
56     printf("\\n");
```

데이터 크기 계산(28~29 라인)

패킷의 전체 캡처 길이 - 모든 헤더의 총 길이 = 실제 전송된 HTTP 데이터의 크기

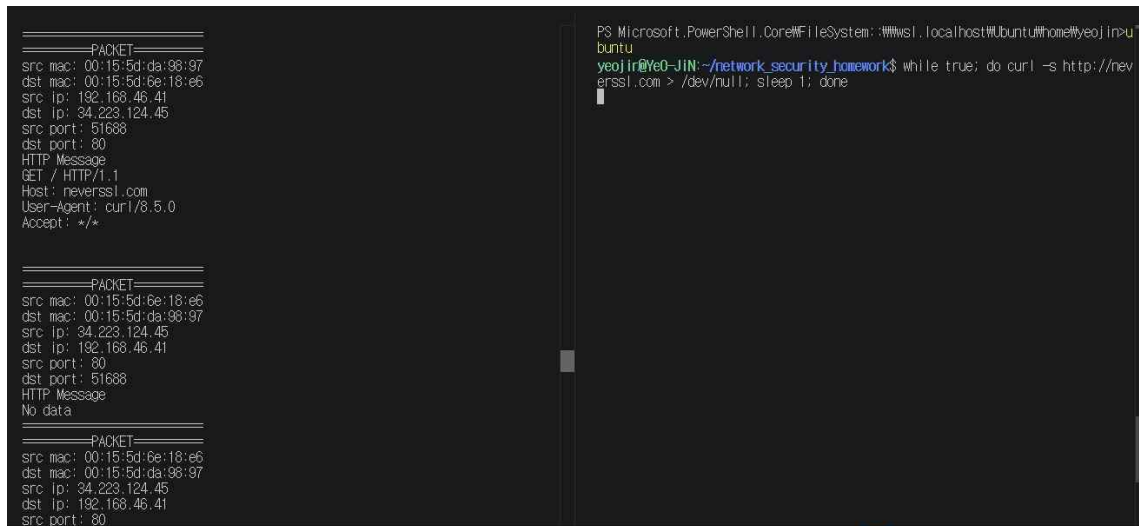
data_len > 0일 경우(데이터가 존재하는 경우)에만 루프를 돌며 isprint() 함수를 통해 출력 가능한 문자(알파벳, 숫자, 기호 등)와 줄바꿈 문자(\\n, \\r)만 가독성 있게 터미널에 화면 출력하도록 제어한다.

```

57     }
58     else{
59         printf("No data\n");
60     }
61     printf("=====\n");
62 }
63 }
64 }
65
66 int main()
67 {
68     pcap_t *handle;
69     char errbuf[PCAP_ERRBUF_SIZE];
70     struct bpf_program fp;
71     char filter_exp[] = "tcp";
72     bpf_u_int32 net;
73
74     // Step 1: Open live pcap session on NIC with name enp0s3
75     handle = pcap_open_live("eth0", BUFSIZ, 1, 1000, errbuf);
76
77     // Step 2: Compile filter_exp into BPF psuedo-code
78     pcap_compile(handle, &fp, filter_exp, 0, net);
79     if (pcap_setfilter(handle, &fp) !=0) {
80         pcap_perror(handle, "Error:");
81         exit(EXIT_FAILURE);
82     }
83
84     // Step 3: Capture packets
85     pcap_loop(handle, -1, got_packet, NULL);
86
87     pcap_close(handle); //Close the handle
88     return 0;
89 }

```

sniff.c 파일을 컴파일 한 후 `sudo ./sniff`를 통해 실행시키면 패킷을 기다리는 대기 상태가 된다. 다른 터미널 창을 열어 트래픽을 발생시키면 패킷 정보가 대기 상태였던 터미널에 뜨는 것을 확인할 수 있다.



```
=====
--PACKET--
src mac: 00:15:5d:da:98:97
dst mac: 00:15:5d:6e:18:e6
src ip: 192.168.46.41
dst ip: 34.223.124.45
src port: 51688
dst port: 80
HTTP Message
GET / HTTP/1.1
Host: neverssl.com
User-Agent: curl/8.5.0
Accept: */*

=====
--PACKET--
src mac: 00:15:5d:6e:18:e6
dst mac: 00:15:5d:da:98:97
src ip: 34.223.124.45
dst ip: 192.168.46.41
src port: 80
dst port: 51688
HTTP Message
No data

=====
--PACKET--
src mac: 00:15:5d:6e:18:e6
dst mac: 00:15:5d:da:98:97
src ip: 34.223.124.45
dst ip: 192.168.46.41
src port: 80

PS Microsoft.PowerShell.Core\FileSystem::\\wsl.localhost\Ubuntu\home\yeojin\p
buntu
yeojin@Ye0-JIN:~/network_security_homework$ while true; do curl -s http://nev
erssl.com > /dev/null; sleep 1; done
```

HTTP Request 분석

src mac / dst mac: 내 PC의 MAC 주소와 게이트웨이(라우터)의 MAC 주소가 정상 출력됨.
src ip: 192.168.46.41 → dst ip: 34.223.124.45
src port: 51688 (임시 포트) → dst port: 80 (HTTP 기본 포트)
HTTP Message 내용: GET / HTTP/1.1, Host: neverssl.com, User-Agent: curl/8.5.0 등

분석: Ubuntu 환경(192.168.46.41)에서 curl 명령어를 통해 neverssl.com 서버의 80번
포트로 HTTP GET 요청을 보내는 패킷이다. 암호화되지 않은 텍스트 형태의 HTTP 요청
헤더 정보가 데이터 영역(Payload)에서 깨짐 없이 그대로 노출 및 파싱되는 것을 확인하였다.

HTTP Response 분석

src ip: 34.223.124.45 → dst ip: 192.168.46.41
src port: 80 → dst port: 51688
HTTP Message: No data

분석: neverssl.com 서버(34.223.124.45) 측에서 내 PC로 전달되는 TCP 패킷이다.
파일로드(HTTP 데이터) 영역이 비어 있어 No data로 출력되었는데, 이는 TCP 연결을
제어하기 위한 세션 제어 패킷(Pure ACK 등)이거나 분할된 패킷의 일부이기 때문에 데이터
크기가 0으로 정확히 계산된 결과이다.

결론

본 과제를 통해 IP 헤더와 TCP 헤더의 길이가 가변적이라는 점을 고려하여 포인터 연산으로 Application 계층의 데이터를 추출하는 방법을 실습하였다. 또한 일반 HTTP 프로토콜(neverssl.com)을 사용할 경우, 패킷 스니핑을 통해 Host, User-Agent와 같은 HTTP 요청 헤더가 암호화되지 않은 채 그대로 노출되는 것을 확인할 수 있었다. 이를 통해 네트워크에서 데이터의 기밀성을 보호하기 위해서는 HTTPS(SSL/TLS)를 적용하는 것이 중요하다는 점을 이해할 수 있었다.